Code Contributing Guidlines

Branching Guidelines, Naming Conventions, and Code Review Process

To maintain a clean and efficient workflow, please adhere to the following branching strategy, naming conventions, and code review processes based on **Trunk-Based Development**. This approach emphasizes frequent integration into the `main` branch, minimizes merge conflicts, and ensures a stable codebase ready for deployment at any time. Incorporating code reviews through pull requests ensures high code quality and knowledge sharing among team members.

Branching Strategy

In trunk-based development, all developers work closely on a single branch, typically the `main` branch. Short-lived feature branches are used but should be merged back into `main` as quickly as possible after going through a code review process via pull requests.

Main Branch

- `main`: The primary branch containing the latest production-ready code. All changes are integrated here promptly after passing code reviews.

Short-Lived Feature Branches

- **Purpose**: For isolated development of small features or fixes that can't be committed directly to `main` immediately.
- **Branch off from**: `main`
- **Merge into**: `main` via pull requests
- **Lifecycle**: Should be short-lived (ideally less than a day or two).

Branch Naming Conventions

Feature Branches

- **Naming Convention**: `feature/<ticket-number>-<brief-description>`
- **Examples**:

- `feature/proj-123-add-user-profile`
- `feature/proj-456-update-dashboard-ui`

Bugfix Branches

- **Naming Convention:** `fix/<ticket-number>-<brief-description>`
- **Examples:**
  - `fix/proj-789-fix-login-error`
  - `fix/proj-101-adjust-timezone-issue`

Experiment Branches

- **Naming Convention:** `experiment/<ticket-number>-<brief-description>`
- **Examples:**
  - `experiment/proj-202-test-new-api`
  - `experiment/proj-303-optimize-loading`

Workflow Guidelines

1. Creating a Short-Lived Feature or Bugfix Branch

- **Branch from** `main`
  - `git checkout main`
  - `git pull origin main`
  - `git checkout -b feature/brief-description`
    `# or for bugfix`
  - `git checkout -b fix/brief-description`

2. Developing and Committing Changes

- Make your changes and commit frequently.
- Follow commit message conventions (e.g., Conventional Commits).
  - `git add .`
  - `git commit -m "feat: implement search functionality"`

3. Keeping Your Branch Updated

- **Rebase Frequently to Incorporate Latest Changes from `maing it fetch origin`**
  - `git rebase origin/main`

4. Opening a Pull Request for Code Review

- **Push Your Branch to Remote**
  - `git push origin feature/brief-description`
- **Open a Pull Request (PR)**
  - Navigate to your repository on GitHub/GitLab/etc.
  - Open a new PR from your feature/bugfix branch into `main`.
  - Provide a clear description of the changes and reference any related issues.
- **Code Review Process**
  - Assign reviewers or request reviews from team members.
  - Reviewers should:
    - Check for code correctness, style adherence, and potential issues.
    - Provide constructive feedback or approve the PR.
  - Address any feedback by making additional commits to your branch.

5. Merging the Pull Request

- **After Approval**
  - Ensure all checks/tests pass.
  - Merge the PR into `main` using the "Squash and Merge" or "Rebase and Merge" option to keep history clean.
  - **Note**: Avoid "Merge Commit" to maintain a linear history.
- **Delete the Feature Branch**
  - After merging, delete the feature branch both remotely and locally.
  - `git branch -d feature/brief-description`
  - `git push origin --delete feature/brief-description`

6. Continuous Integration

- **Automated Testing**
  - Every PR and commit to `main` should trigger automated tests.
  - Fix any broken builds immediately.

Code Review Guidelines

- **Timeliness**
  - Review PRs promptly to keep the development process moving.
- **Quality Feedback**
  - Provide clear, specific, and constructive feedback.
  - Focus on the code and not the coder.
- **Approval Criteria**
  - Code adheres to style guidelines and conventions.
  - All tests pass, and new tests are added if necessary.
  - The code solves the intended problem without introducing new issues.
- **Resolving Conflicts**
  - If there are merge conflicts, rebase your branch onto `main` and resolve conflicts.

Pull Request Guidelines

PR Naming Convention

Format: `[type]: Brief description`

- Examples:
  - `feat: Add user auth`
  - `fix: Resolve login issue`

PR Description Template

```
### Changes
- What changed?
- Why did it change?
```

```
### Related
- Ticket: PROJ-XXX

### Testing
- How was it tested?
```

Additional Guidelines

- **Feature Flags**
  - Use feature toggles to control the activation of new features without long-lived branches.
  - Allows incomplete features to exist in `main` without affecting production.
- **Commit Messages**
  - Follow the Conventional Commits specification (https://www.conventionalcommits.org/).
  - Include the ticket number in the commit message body or footer.
  - Examples: `feat(user): add user profile page`
    `[proj-123] Implement user profile page with editable fields and avatar upload.fix(auth): correct login validation error`
    `[proj-789] Resolve issue with incorrect password validation logic.`
- **Frequent Integration**
  - Integrate changes into `main` multiple times a day via PRs.
  - Avoid large merges by committing small, incremental updates.
- **Collaborative Development**
  - Communicate with the team about ongoing work to prevent overlap.
  - Pair programming or mob programming can be effective.

Examples of Branch Usage with Code Reviews

Implementing a New Feature

1. **Create Branch**

```
git checkout main
git pull origin main
git checkout -b feature/proj-404-improve-performance
```

2. **Develop Feature**

```
git add .
git commit -m "feat(performance): optimize database queries
[proj-404] Implement caching layer for frequently accessed data."
```

- Make changes and commit frequently using Conventional Commits.

3. **Rebase and Push**

```
git fetch origin
git rebase origin/main
git push origin feature/proj-404-improve-performance
```

4. **Open a Pull Request**

- Go to your repository and open a PR from `feature/proj-404-improve-performance` into `main`.
- Example PR: `feat: Add user auth`

```
### Changes
- Added user authentication endpoints
- Implemented JWT token validation
- Created middleware for protected routes

### Related
- Ticket: PROJ-123

### Testing
- Unit tests added for auth endpoints
- Manual testing with Postman
- Integration tests for middleware
```

- Request reviews from team members.

5. **Code Review**

- Address feedback by committing changes to your branch.
- Reviewers approve the PR once satisfied.

6. **Merge PR**

- After approval and passing tests, merge the PR into `main`.
- Delete the feature branch.

Fixing a Bug

1. **Create Bugfix Branch** `git checkout main`
   ```
   git pull origin main
   git checkout -b fix/proj-505-resolve-crash-on-startup
   ```
2. **Fix Bug and Commit** `git add .`
   ```
   git commit -m "fix(startup): resolve crash when application starts
   [proj-505] Handle null configuration values during initialization."
   ```
3. **Push and Open PR** `git push origin fix/proj-505-resolve-crash-on-startup`
   - Open a PR into `main` and request a quick review.
4. **Code Review**
   - Respond to any feedback promptly.
5. **Merge PR**
   - Merge into `main` after approval and passing tests.
   - Delete the bugfix branch.